

ΕΡΓΑΣΙΑ ΨΗΦΙΑΚΑ ΣΥΣΤΗΜΑΤΑ

HW-1

ΜΙΧΑΗΛ ΜΠΑΖΙΑΚΟΣ

ΑΕΜ:11125

Άσκηση 1

Στην Άσκηση 1 υλοποιήθηκε μια Αριθμητική και Λογική Μονάδα (ALU) 32-bit, η οποία εκτελεί προσημασμένες αριθμητικές πράξεις (πρόσθεση, αφαίρεση, πολλαπλασιασμό), λογικές πράξεις (AND, OR, XOR, NAND, NOR) και πράξεις ολίσθησης (λογικές και αριθμητικές).

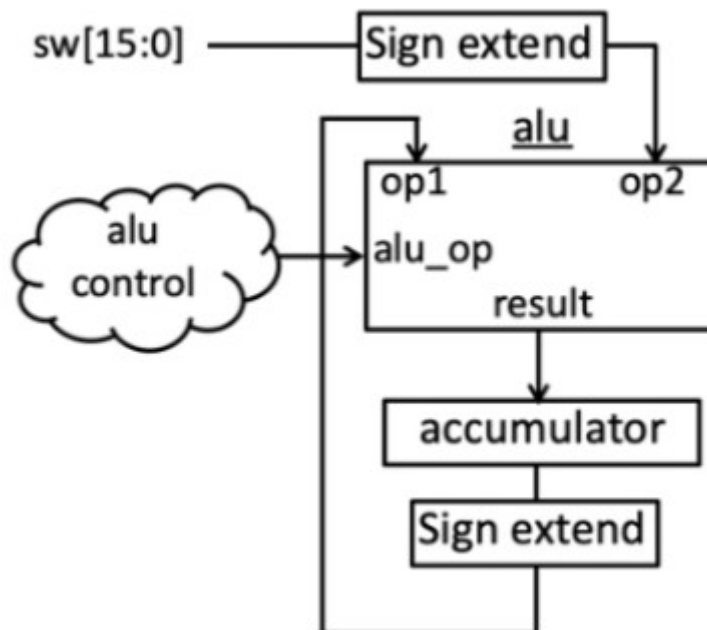
Το κύκλωμα **alu.v** ορίστηκε παραμετρικά με SIZE=32 και διαθέτει τρεις εισόδους: op1, op2 (τελεστές) και alu_op (κωδικός πράξης), καθώς και τρεις εξόδους: result, zero και onf. Για την επιλογή της πράξης χρησιμοποιήθηκε η εντολή case εντός ενός always @(*) block, καθιστώντας το κύκλωμα αμιγώς συνδυαστικό.

Για την ορθή εκτέλεση των αριθμητικών πράξεων και της αριθμητικής ολίσθησης (>>>), έγινε χρήση της συνάρτησης \$signed() της Verilog, ώστε οι τελεστές να ερμηνεύονται ως αριθμοί συμπληρώματος ως προς 2.

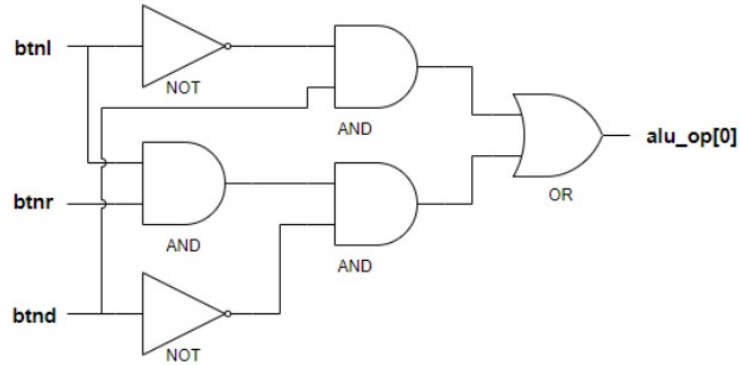
Η ανίχνευση υπερχείλισης (overflow) υλοποιήθηκε μέσω ενός προσωρινού καταχωρητή full_product διπλάσιου εύρους (64-bit). Το onf ενεργοποιείται όταν τα ανώτερα 32 bits του full_product δεν συμφωνούν με την επέκταση προσήμου του τελικού αποτελέσματος των 32 bits. Τέλος, η έξοδος zero ενεργοποιείται λογικά όταν το τελικό result είναι ίσο με μηδέν.

Άσκηση 2

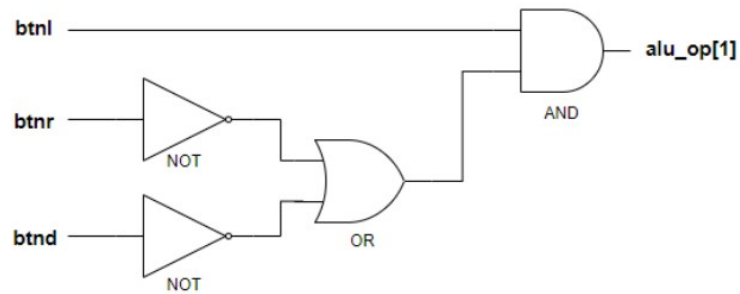
Στην Άσκηση 2 υλοποιήθηκε ένα κύκλωμα αριθμομηχανής, το οποίο αξιοποιεί την ALU που σχεδιάστηκε στο προηγούμενο ερώτημα. Το κύκλωμα συνδυάζει τα πλήκτρα εισόδου (btnc, btnac, btnl, btnr, btnd) για τον καθορισμό της λειτουργίας της ALU και τον έλεγχο του συσσωρευτή (accumulator). Η υλοποίηση οργανώθηκε σε τρία επιμέρους αρχεία: το calc_enc.v που αποτελεί το συνδυαστικό κύκλωμα ελέγχου, το calc.v που είναι η κύρια μονάδα με τη σύγχρονη λογική, και το tb_calc.v για τον έλεγχο της λειτουργίας.



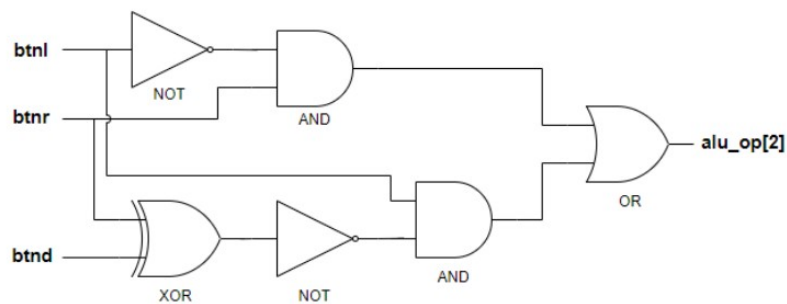
Το κύκλωμα **calc_enc.v** υλοποιήθηκε με χρήση Structural Verilog (δομική περιγραφή), ακολουθώντας πιστά τα λογικά διαγράμματα των Σχημάτων 2 έως 5 της εκφώνησης . Η μονάδα αυτή δέχεται ως εισόδους τα πλήκτρα κατεύθυνσης (btnl, btnr, btnd) και, μέσω πυλών λογικής, παράγει τον τετράμπιτο κωδικό πράξης alu_op, ο οποίος καθορίζει τη λειτουργία που θα εκτελέσει η ALU .



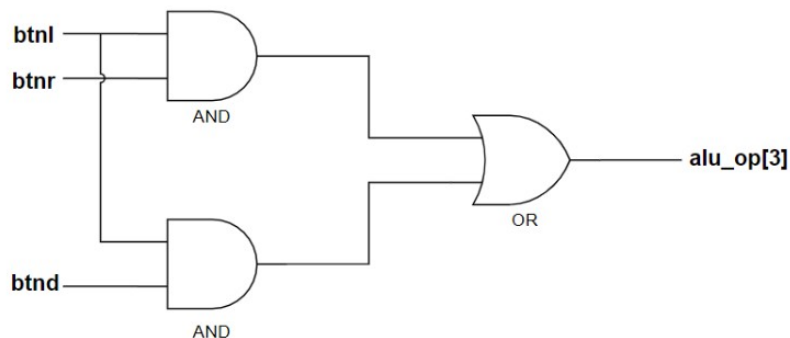
Σχ. 2: Παραγωγή του alu_op[0] μέσω των btnr, btnl, btnd.



Σχ. 3: Παραγωγή του alu_op[1] μέσω των btnr, btnl, btnd.



Σχ. 4: Παραγωγή του alu_op[2] μέσω των btnr, btnl, btnd.



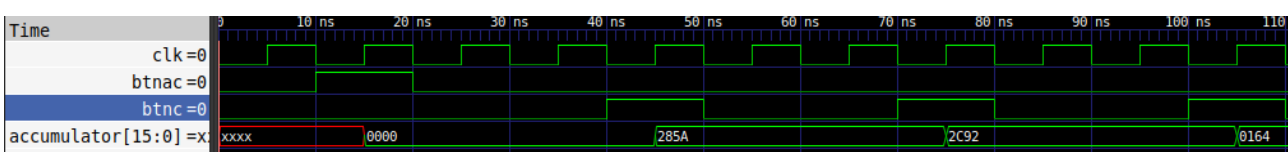
Σχ. 5: Παραγωγή του alu_op[3] μέσω των btnr, btnl, btnd.

Η κύρια μονάδα **calc.v** υλοποιεί το διάγραμμα ροής του συστήματος, συνδέοντας την έξοδο του κωδικοποιητή με την είσοδο ελέγχου της ALU . Αναφορικά με τη διαχείριση των δεδομένων, οι είσοδοι των διακοπών (sw) και του συσσωρευτή (accumulator), οι οποίες έχουν εύρος 16-bit, υφίστανται επέκταση προσήμου (sign extension) σε 32-bit πριν εισαχθούν στην ALU, ώστε να διασφαλιστεί η ορθότητα των προσημασμένων πράξεων . Η ενημέρωση του αποτελέσματος πραγματοποιείται σύγχρονα με το ρολόι (clk) εντός ενός always block . Συγκεκριμένα, το πλήκτρο btnac λειτουργεί ως σύγχρονο Reset που μηδενίζει τον accumulator , ενώ το btnc (central button) ενεργοποιεί την αποθήκευση του αποτελέσματος της ALU στον καταχωρητή . Τέλος, η τρέχουσα τιμή του accumulator οδηγείται συνεχώς στην έξοδο led μέσω εντολής συνεχούς ανάθεσης (continuous assignment), επιτρέποντας την οπτική απεικόνιση του αποτελέσματος .

Testbench

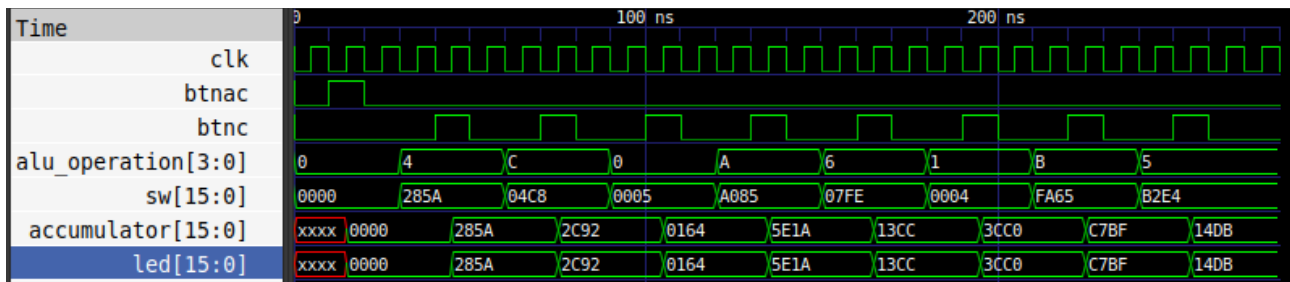
btnl, btnr, btnd (input)	Previous value (acc.)	Switches (input)	Function in ALU	Expected Result
(btnac for reset)	xxxx	xxxx	Reset	0x0
0,1,0	0x0	0x285a	ADD	0x285a
1,1,1	0x285a	0x04c8	XOR	0x2c92
0,0,0	0x2c92	0x0005	Logical Shift Right	0x0164
1,0,1	0x0164	0xa085	NOR	0x5e1a
1,0,0	0x5e1a	0x07fe	MULT	0x13cc
0,0,1	0x13cc	0x0004	Logical Shift Left	0x3cc0
1,1,0	0x3cc0	0xfa65	NAND	0xc7bf
0,1,1	0xc7bf	0xb2e4	SUB	0x14db

Τέλος για την επαλήθευση της ορθής λειτουργίας του κυκλώματος αναπτύχθηκε το testbench **tb_calc.v**, το οποίο παράγει ένα σήμα ρολογιού με περίοδο 10ns . Η διαδικασία ελέγχου ξεκινά με την ενεργοποίηση του σήματος επαναφοράς btnac τη χρονική στιγμή 10ns, εξασφαλίζοντας τον σύγχρονο μηδενισμό του συσσωρευτή στην πρώτη θετική ακμή του ρολογιού . Στη συνέχεια, ακολουθεί η σειρά δοκιμών που ορίζει η εκφώνηση, όπου για κάθε πράξη τίθενται αρχικά οι τιμές των διακοπών sw και των πλήκτρων επιλογής (btnl, btnr, btnd) . Αφού σταθεροποιηθούν οι είσοδοι, ενεργοποιείται το κεντρικό πλήκτρο btnc για χρονικό διάστημα 10ns, ώστε να συμπίσει με την ανερχόμενη ακμή του ρολογιού και να επιτευχθεί η σύγχρονη εγγραφή του αποτελέσματος στον accumulator . Όπως επιβεβαιώνεται και από τις κυματομορφές εξόδου, όλες οι μεταβάσεις και τα τελικά αποτελέσματα εμφανίζονται στις αναμενόμενες χρονικές στιγμές, επαληθεύοντας την ορθότητα της σχεδίασης.



Ο accumulator μηδενίζεται **σύγχρονα** με το πάτημα του btnac και ενημερώνεται **σύγχρονα** κάθε φορά που πατιέται το "central"(btnc).

Οι κυματομορφές του testbench



Στο παρόν σχήμα απεικονίζονται τα αποτελέσματα της προσομοίωσης για το σύνολο των ζητούμενων πράξεων, όπως αυτές καθορίστηκαν στον πίνακα ελέγχου της εκφώνησης. Αρχικά, διακρίνεται η επιτυχής επαναφορά του συστήματος μέσω του σήματος btnac, όπου ο συσσωρευτής (accumulator) μηδενίζεται σύγχρονα στην πρώτη θετική ακμή του ρολογιού. Στη συνέχεια, η ροή εκτέλεσης ακολουθεί διαδοχικά τα σενάρια ελέγχου, όπου για κάθε πράξη απεικονίζονται η τιμή εισόδου των διακοπτών (sw), ο υπολογιζόμενος κωδικός πράξης (alu_operation) και το τελικό αποτέλεσμα. Παρατηρείται ότι η ενημέρωση του συσσωρευτή και της εξόδου led πραγματοποιείται αποκλειστικά κατά τη θετική ακμή του σήματος clk και μόνο εφόσον είναι ενεργοποιημένο το πλήκτρο btnc, επιβεβαιώνοντας την ορθή σύγχρονη συμπεριφορά του κυκλώματος. Τέλος, οι δεκαεξαδικές τιμές εξόδου (π.χ. 285A, 2C92, 0164 κ.ο.κ.) ταυτίζονται απόλυτα με τα αναμενόμενα αποτελέσματα, επαληθεύοντας την ορθότητα της ALU και του κυκλώματος ελέγχου.

Η έξοδος απο το test-bench που είναι ίδια με αυτή της εκφώνησης:

```

Starting Calculator Testbench...
VCD info: dumpfile tb_calc.vcd opened for output.
Time=0 | Sw=0000 | BTN(L,R,D)=000 | Acc=xxxx
Time=15000 | Sw=0000 | BTN(L,R,D)=000 | Acc=0000
Time=30000 | Sw=285a | BTN(L,R,D)=010 | Acc=0000
Time=45000 | Sw=285a | BTN(L,R,D)=010 | Acc=285a
Time=60000 | Sw=04c8 | BTN(L,R,D)=111 | Acc=285a
Time=75000 | Sw=04c8 | BTN(L,R,D)=111 | Acc=2c92
Time=90000 | Sw=0005 | BTN(L,R,D)=000 | Acc=2c92
Time=105000 | Sw=0005 | BTN(L,R,D)=000 | Acc=0164
Time=120000 | Sw=a085 | BTN(L,R,D)=101 | Acc=0164
Time=135000 | Sw=a085 | BTN(L,R,D)=101 | Acc=5e1a
Time=150000 | Sw=07fe | BTN(L,R,D)=100 | Acc=5e1a
Time=165000 | Sw=07fe | BTN(L,R,D)=100 | Acc=13cc
Time=180000 | Sw=0004 | BTN(L,R,D)=001 | Acc=13cc
Time=195000 | Sw=0004 | BTN(L,R,D)=001 | Acc=3cc0
Time=210000 | Sw=fa65 | BTN(L,R,D)=110 | Acc=3cc0
Time=225000 | Sw=fa65 | BTN(L,R,D)=110 | Acc=c7bf
Time=240000 | Sw=b2e4 | BTN(L,R,D)=011 | Acc=c7bf
Time=255000 | Sw=b2e4 | BTN(L,R,D)=011 | Acc=14db
Testbench Completed.

```

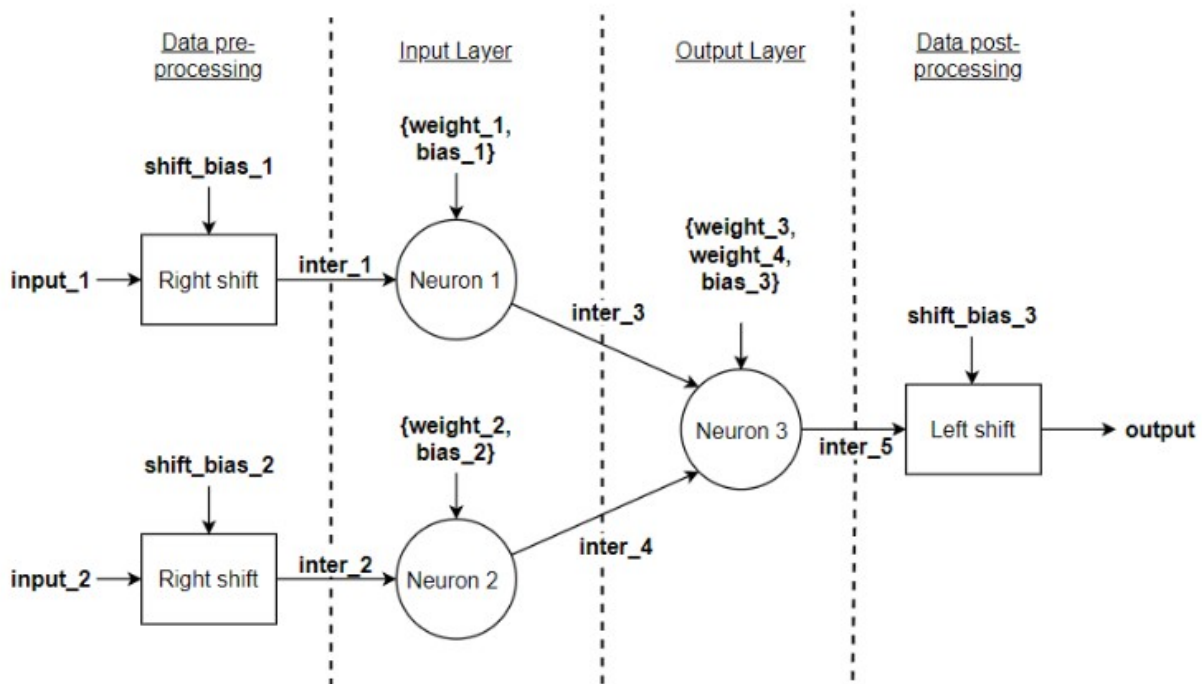
Άσκηση 3

Στην Άσκηση 3 υλοποιήθηκε ένα αρχείο καταχωρητών (Register File) το οποίο επιτρέπει την ταυτόχρονη ανάγνωση και εγγραφή δεδομένων. Το κύκλωμα ορίζει εσωτερικά έναν πίνακα 16 καταχωρητών των 32-bit (registers [15:0]). Η λειτουργία του κυκλώματος βασίζεται αρχικά σε σύγχρονη λογική μέσω ενός always block, όπου ελέγχεται η εγγραφή και η επαναφορά. Το σήμα resetn λειτουργεί ασύγχρονα και είναι ενεργό σε αρνητική ακμή (active low), μηδενίζοντας όλους τους καταχωρητές, ενώ η εγγραφή πραγματοποιείται σύγχρονα με το ρολόι. Συγκεκριμένα, όταν το σήμα write είναι ενεργό, τα δεδομένα από τις εισόδους writeData1 και writeData2 αποθηκεύονται στις αντίστοιχες διευθύνσεις writeReg1 και writeReg2.

Παράλληλα, η ανάγνωση υλοποιείται μέσω συνδυαστικής λογικής σε ένα ξεχωριστό always block, διαθέτοντας 4 θύρες ανάγνωσης. Για την αποφυγή σφαλμάτων συνοχής δεδομένων, υλοποιήθηκε μηχανισμός προώθησης (forwarding), δίνοντας προτεραιότητα στα νέα δεδομένα σε περίπτωση που ζητηθεί ανάγνωση από διεύθυνση στην οποία ταυτόχρονα πραγματοποιείται εγγραφή. Αυτό επιτυγχάνεται με εντολές ελέγχου όπως if (write && (readReg1 == writeReg1)), εξασφαλίζοντας ότι η έξοδος αντικατοπτρίζει πάντα την πιο πρόσφατη τιμή.

Άσκηση 4

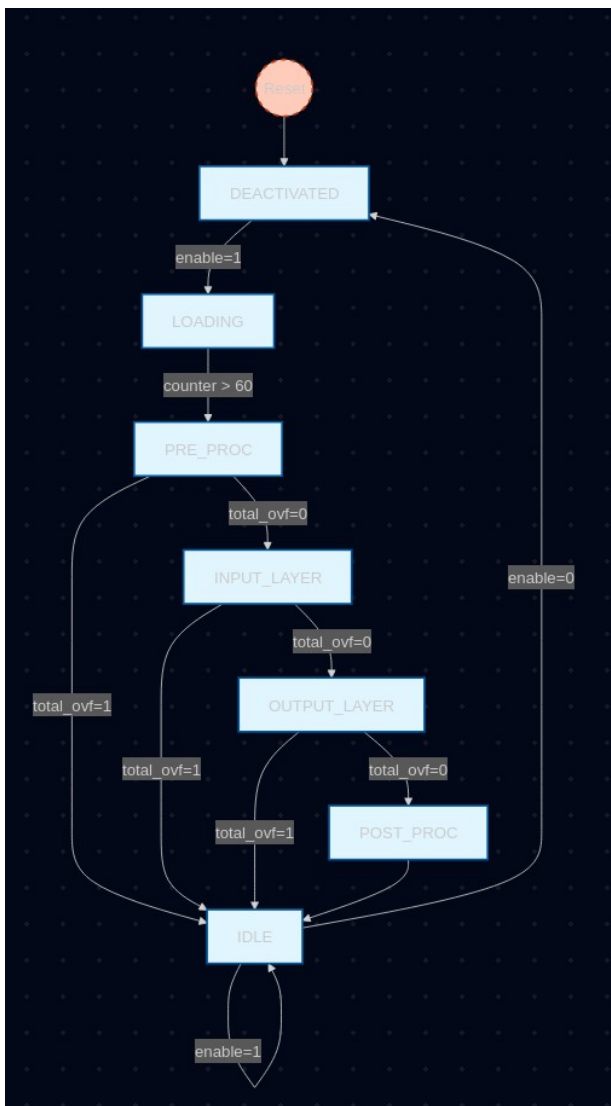
Στην Άσκηση 4 υλοποιήθηκε ένα κύκλωμα νευρωνικού δικτύου, βασισμένο στο σχήμα 6, καθώς και ο ελεγκτής του (FSM 4 επιπέδων). Το κύκλωμα συνδυάζει την ALU, το Regfile και το νέο κύκλωμα MAC. Η λειτουργία του χωρίζεται σε 7 στάδια: DEACTIVATED, LOADING, PRE_PROC, INPUT_LAYER, OUTPUT_LAYER, POST_PROC και IDLE. Στο στάδιο DEACTIVATED αρχικοποιείται το νευρωνικό, ενώ στο LOADING γεμίζει το Regfile με τις τιμές της ROM βάσει του load_counter, μέχρι αυτός να φτάσει το 60. Ακολούθως, στο PRE_PROC εκτελείται δεξιά ολίσθηση (right shift) στα δύο inputs. Στο INPUT_LAYER πραγματοποιούνται οι πράξεις $inter_3 = inter_1 * weight_1 + bias_1$ και $inter_4 = inter_2 * weight_2 + bias_2$, ενώ στο OUTPUT_LAYER υπολογίζεται το $inter_5 = inter_3 * weight_3 + inter_4 * weight_4 + bias_3$. Στο POST_PROC γίνεται αριθμητική ολίσθηση στα αριστερά μέσω της ALU κατά shift_bias_3. Τέλος, στο IDLE μεταβαίνουμε όταν ολοκληρωθεί η διαδικασία, δίνοντας στην έξοδο τη μέγιστη αρνητική τιμή αν υπάρχει υπερχειλίση, και αναμένοντας το σήμα enable. **Η άσκηση δομείται σε 3 επίπεδα: το κύκλωμα mac, το νευρωνικό και το testbench.**



Σχ. 6: Απλό νευρωνικό σύστημα με τρεις νευρώνες.

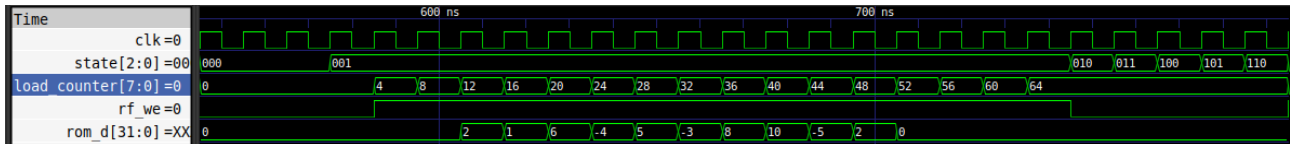
Στο κύκλωμα **mac_unit.v** υλοποιείται η πράξη πολλαπλασιασμού μιας εισόδου (op1) με το αντίστοιχο βάρος (op2) και ακολουθεί η πρόσθεση της εκάστοτε πόλωσης (op3). Σε περίπτωση που συμβεί μηδενισμός ή υπερχειλίση, ενεργοποιούνται τα αντίστοιχα flags (zero_mul, zero_add, ovf_mul, ovf_add).

Στο κύκλωμα **nn.v** υλοποιείται το νευρωνικό δίκτυο στα 7 παραπάνω στάδια με τη χρήση τεσσάρων always blocks. Το πρώτο block ενημερώνει μόνο την κατάσταση (state) χρησιμοποιώντας non-blocking εντολές με το ρολόι, ενώ το δεύτερο υπολογίζει αποκλειστικά την επόμενη κατάσταση (next_state) χρησιμοποιώντας blocking εντολές. Το τρίτο block υπολογίζει τα σήματα ελέγχου και τις εισόδους των modules (ALU, Muxes) χρησιμοποιώντας επίσης blocking εντολές και, τέλος, το τέταρτο block ενημερώνει τους καταχωρητές δεδομένων, αλλά όχι την κατάσταση του FSM, χρησιμοποιώντας non-blocking εντολές. Η συγκεκριμένη σχεδίαση ικανοποιεί πλήρως τις προϋποθέσεις διαχωρισμού της ακολουθιακής από τη συνδυαστική λογική. Ο τύπος του FSM είναι μηχανή Moore, διότι τα σήματα ελέγχου (π.χ., alu_ctrl, rf_r1, rf_we) εξαρτώνται αποκλειστικά από την τρέχουσα κατάσταση και όχι άμεσα από τις εισόδους. Επιλέχθηκε διότι προσφέρει μεγάλη ευστάθεια έναντι πιθανών θορύβων, ενώ ο σταδιακός υπολογισμός του νευρωνικού ευνοεί τη δομή Moore. Η αποθήκευση δεδομένων γίνεται με χρήση ενδιάμεσων καταχωρητών (inter_1, inter_2, κλπ.) για λόγους ταχύτητας, ώστε να μην απαιτείται εγγραφή πίσω στο Regfile, για μείωση της πολυπλοκότητας λόγω αποφυγής πολύπλοκης διαχείρισης διευθύνσεων και λόγω της ροής δεδομένων, καθώς, αφού το νευρωνικό είναι Feed-Forward, τα ενδιάμεσα δεδομένα χρειάζονται μόνο μία φορά και κατόπιν απορρίπτονται. Ακολουθεί διάγραμμα FSM για καλύτερη εξήγηση των σταδίων μετάβασης.



**FSM διάγραμμα
κυκλώματος nn.v**

Τέλος, στο επίπεδο του **tb_nn.v**, υλοποιείται η αυτοματοποιημένη επαλήθευση της σωστής λειτουργίας του νευρωνικού δικτύου. Το testbench παράγει τα απαραίτητα σήματα ρολογιού (clk) και επαναφοράς (resetsn), ενώ τροφοδοτεί το κύκλωμα με τυχαία διανύσματα εισόδου (input_1, input_2) μέσω βρόχου επανάληψης. Η εγκυρότητα των αποτελεσμάτων ελέγχεται δυναμικά, συγκρίνοντας την έξοδο του σχεδιασμένου υλικού (final_output) με τις τιμές που παράγει ένα πρότυπο μοντέλο αναφοράς (nn_model). Η διαδικασία ελέγχου καλύπτει τον πλήρη κύκλο λειτουργίας της FSM (από την κατάσταση DEACTIVATED έως την IDLE), καθώς και τις οριακές περιπτώσεις υπερχείλισης (total_ovf), εκτυπώνοντας τα αποτελέσματα επιτυχίας ή αποτυχίας στην κονσόλα προσομοίωσης.



Στην κυματομορφή που απεικονίζει την κατάσταση LOADING, επιβεβαιώνεται η ορθή λειτουργία του μηχανισμού μεταφοράς δεδομένων από τη μνήμη ROM στο Register File. Παρατηρούμε ότι ο μετρητής load_counter αυξάνεται με βήμα 4 σε κάθε ακμή του ρολογιού (0, 4, 8...). Η τιμή αυτή οδηγεί σωστά τη διεύθυνση εγγραφής rf_w (load_counter >> 2), εξασφαλίζοντας ότι τα δεδομένα rom_d αποθηκεύονται διαδοχικά στους καταχωρητές. Το σήμα rf_we (Write Enable) παραμένει ενεργό (High) καθ' όλη τη διάρκεια της φάσης, επιτρέποντας την εγγραφή. Η διαδικασία τερματίζεται επιτυχώς μόλις ο μετρητής ξεπεράσει το όριο των 60, οπότε και η FSM μεταβαίνει αυτόματα στην επόμενη κατάσταση PRE_PROC, ολοκληρώνοντας την αρχικοποίηση του δικτύου.



Στην κυματομορφή για τα πρώτα 3 test case (i=0 πρώτο με εύρος τιμών [-4096, 4095] με 2 overflow [max_positive_number / 2 - max_positive_number] και [max_negative_number - max_negative_number / 2]), παρατηρούμε την επιτυχή επαλήθευση της λειτουργίας του νευρωνικού δικτύου μέσω της σύγκρισης της εξόδου του υλικού με το πρότυπο μοντέλο αναφοράς και της μετάβασης στο IDLE state με το που εμφανιστεί overflow. Αρχικά, διαπιστώνεται ότι το σήμα ref_output ενημερώνεται ακαριαία με την εφαρμογή των εισόδων, καθώς προέρχεται από τη συνάρτηση συμπεριφοράς του testbench, σε αντίθεση με το final_output το οποίο παρουσιάζει την αναμενόμενη χρονική υστέρηση λόγω της διαδοχικής εκτέλεσης των σταδίων της FSM (Loading, Pre-processing, Layers). Η ορθότητα της σχεδίασης επιβεβαιώνεται τη στιγμή που η μηχανή καταστάσεων εισέρχεται στην κατάσταση IDLE, όπου η τιμή του final_output σταθεροποιείται και ταυτίζεται απόλυτα με το ref_output σε μορφή προσημασμένου δεκαδικού. Η ταύτιση αυτή, σε συνδυασμό με την προσημασμένη απεικόνιση, αποδεικνύει ότι η ροή δεδομένων και οι αριθμητικές πράξεις συμπληρώματος ως προς 2 στις μονάδες MAC και ALU εκτελούνται σωστά, επαληθεύοντας τη λειτουργική ακρίβεια του κυκλώματος έναντι του θεωρητικού μοντέλου.